



Reflecta

FACHKONZEPT · ADMIN-BEREICH

Fachkonzept des Admin-Bereichs

Dialogliste, Informationsarchitektur, Swimlane-Ablaufdiagramm
und annotierter Wireframe des Admin-Dashboards der Reflecta-
Plattform

PROJEKT

Reflecta

DATUM

22.05.2026

DEPARTMENT

Frontend-
Adminforms

MITARBEITER

8533720

DOKUMENTÜBERSICHT

Inhalt

Dieses Dokument spezifiziert den Admin-Bereich von Reflecta. Er ist ausschließlich für Nutzer mit `is_admin = true` zugänglich und stellt Betriebskennzahlen der Plattform dar. Das Fachkonzept beschreibt den einzigen Dialog des Bereichs, dessen Informationsarchitektur, den Ablauffluss sowie einen annotierten Wireframe.

1	Geltungsbereich & Konventionen	3
2	Dialogliste	4
3	Informationsarchitektur	5–6
4	Swimlane-Ablaufdiagramm	7–8
4.1	Flow A — Stats-Übersicht laden (Erfolgsfall)	7
4.2	Flow B — Zugriff verweigert (HTTP 403 / 401)	8
5	Wireframe	9
5.1	ADM-00 Stats Overview	9

ABSCHNITT 1

Geltungsbereich & Konventionen

Der Admin-Bereich ist nur für angemeldete Nutzer mit gesetztem `is_admin`-Flag zugänglich. Er besteht aus einer einzigen Ansicht und bietet keine weiteren Tabs oder Sub-Dialoge.

1.1 Im Geltungsbereich

- **AdminDashboard (ADM-00)** mit vier StatCards und einer AI-Usage-Card.
- **API-Endpunkt** `GET /admin/stats` mit JWT-geschütztem Zugriff.

1.2 Außerhalb des Geltungsbereichs

- Angemeldetes Produkt (Journal, Ziele, Kalender, Analytics) — in einem separaten Fachkonzept dokumentiert.
- Nutzerverwaltung oder manuelle Eingriffe in Daten über das Frontend.
- Authentifizierungsablauf und Token-Ausstellung (AWS Cognito).

1.3 Bezeichner-Konventionen

DIALOG-IDS

ADM-00 kennzeichnet den einzigen Dialog des Admin-Bereichs. Die ID wird in der Dialogliste, im IA-Baum, im Swimlane-Diagramm und im Wireframe konsistent verwendet.

SCHUTZSTUFEN

Gated (Admin) kennzeichnet Dialoge, die ein gültiges JWT mit `is_admin = true` voraussetzen. Alle ADM-Dialoge tragen diese Stufe — andernfalls liefert das Backend HTTP 403.

1.4 Marken- und Typografie-Konventionen

● Primärer Lila-Verlauf #9B30FF → #6A0DAD ● Schrift #171717 ● Helles Lila #F3E8FF ● Erfolg #16A34A

Schriftfamilie ist Plus Jakarta Sans. Display-Gewichte liegen bei 700–800, Fließtext bei 400, Beschriftungen bei 500–600.

1.5 Zugriffsregel

Das Admin-Tab in der Sidebar ist ausschließlich gerendert, wenn der authentifizierte Nutzer `is_admin === true` zurückgibt. Die Route selbst ist nicht direkt ansteuerbar; das Frontend prüft den Flag clientseitig, das Backend serverseitig via `get_current_admin`-Dependency.

ABSCHNITT 2

Dialogliste

Der Admin-Bereich besteht aus einer einzigen Ansicht. Der Dialog-Bezeichner, sein Typ und seine Auslöser sind nachfolgend aufgeführt.

DIALOG-ID	NAME	TYP	AUSLÖSER	BESCHREIBUNG
ADM-00	Stats Overview GATED · ADMIN	Standard-Ansicht	Navigation → Admin	Gesamtübersicht: Nutzer, Einträge, AI-Calls, Tokens

Der Admin-Bereich besteht aktuell aus einer einzigen Ansicht (**AdminDashboard**). Die Seite lädt beim ersten Rendern automatisch via `useEffect` und zeigt die Stats aus `GET /admin/stats`. Es gibt keine weiteren Tabs oder Dialoge.

2.1 Elemente in ADM-00

ELEMENT-ID	NAME	BESCHREIBUNG
ADM-00.1	StatCard — Total Users	Gesamtzahl aller registrierten Nutzer aus <code>stats.total_users</code>
ADM-00.2	StatCard — Active Users (7d)	Nutzer mit mindestens einem Eintrag in den letzten 7 Tagen aus <code>stats.active_users_7d</code>
ADM-00.3	StatCard — Total Entries	Gesamtzahl aller Journaleinträge über alle Nutzer aus <code>stats.total_entries</code>
ADM-00.4	StatCard — AI Calls Today	Anzahl der KI-Aufrufe am heutigen Tag aus <code>stats.ai_calls_today</code>
ADM-00.5	AI Usage Card	Aggregierte KI-Nutzung: Total Calls, Success Rate (%), Total Tokens

ABSCHNITT 3

Informationsarchitektur

Der Admin-Bereich ist eine einzelne, bedarfsgesteuert gemountete Seite innerhalb der AuthenticatedApp. Sie ist **zustandsbehaftet** (stateful): beim ersten Rendern lädt `useEffect` die Statistiken vom Backend und speichert sie via `setStats` im lokalen React-State. Die Hierarchie des Komponentenbaums, der API-Endpunkt und das zugrundeliegende Datenmodell sind nachfolgend dokumentiert.

3.1 Komponentenbaum

App.js

```
└─ AuthenticatedApp [activeTab state]
  │   └─ Sidebar [journal | goals | calendar | analytics | admin]
  │   └─ Header
  │   └─ AdminDashboard ← nur wenn activeTab === "admin"
  │       └─ Lädt via useEffect → getAdminStats() → GET /admin/stats
  │       └─ StatCard: Total Users stats.total_users
  │       └─ StatCard: Active Users (7d) stats.active_users_7d
  │       └─ StatCard: Total Entries stats.total_entries
  │       └─ StatCard: AI Calls Today stats.ai_calls_today
  │       └─ AI Usage Card
  │           └─ Total Calls stats.total_ai_calls
  │           └─ Success Rate (%) stats.ai_success_rate
  │           └─ Total Tokens stats.ai_tokens_total
```

3.2 API-Endpunkt

METHODE	ENDPUNKT	ANTWORT-FELDER	SCHUTZ
GET	/admin/stats	total_users, active_users_7d, total_entries, total_ai_calls, ai_success_rate, ai_calls_today, ai_tokens_total	IS_ADMIN

Der Endpunkt verwendet die `get_current_admin`-Dependency. FastAPI prüft das JWT-Token und stellt sicher, dass `user.is_admin == True` — andernfalls HTTP 403. Intern aggregiert das Backend die Werte ausschließlich über DynamoDB-Queries auf **GS12** (cross-user Counts für User & Entries) und **GS13** (AI-Usage-Aggregation über alle Nutzer). Es gibt keine SQL-Datenbankabfragen.

3.3 Datenmodell (DynamoDB One-Table-Design)

Reflecta nutzt ein **DynamoDB One-Table-Design** — alle Entitäten liegen in einer einzigen Tabelle, unterschieden durch zusammengesetzte PK/SK-Schlüssel. Es gibt keine separaten SQL-Tabellen. Die Admin-Stats werden ausschließlich über Global Secondary Indices (GSI) abgefragt.

Relevante Entitäten & Schlüssel

ENTITÄT	PK	SK	RELEVANTE ATTRIBUTE
User	USER	<sub>	email, is_admin, created_at
JournalEntry	USER#<sub>	ENTRY#<entryId>	created_at
AIUsageLog	USER#<sub>	AILOG#<created_at>#<logId>	feature, success, input_tokens, output_tokens, created_at

Admin-Stats → DynamoDB Access Patterns

STAT-FELD	INDEX	QUERY
total_users	GSI2	GSI2_pk = ENTITY#USER → Count Items
total_entries	GSI2	GSI2_pk = ENTITY#ENTRY → Count Items
active_users_7d	GSI2	GSI2_pk = ENTITY#ENTRY , GSI2_sk BETWEEN <now-7d> AND <now> → Distinct PKs zählen
ai_calls_today	GSI3	GSI3_pk = AILOG#<feature> , GSI3_sk BETWEEN <today> AND <now> → Count Items
total_ai_calls	GSI3	Je Feature: GSI3_pk = AILOG#<feature> → Count Items; Summe über alle Features im Backend aggregiert
ai_success_rate	GSI3	Je Feature: GSI3_pk = AILOG#<feature> , FilterExpression success = true → Count / Gesamt × 100; im Backend berechnet
ai_tokens_total	GSI3	Je Feature: GSI3_pk = AILOG#<feature> → Sum(input_tokens + output_tokens) über alle zurückgegebenen Items; im Backend aggregiert

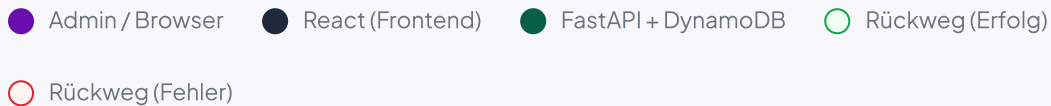
GSI2 (Admin / cross-user lookups): GSI2_pk = ENTITY#<type> , GSI2_sk = <created_at>#<id> — ermöglicht nutzerübergreifendes Zählen aller User- und Entry-Einträge.

GSI3 (AI usage analytics): GSI3_pk = AILOG#<feature> , GSI3_sk = <created_at>#<logId> — ermöglicht zeitraumbasierte Aggregation der KI-Nutzung über alle Nutzer.

ABSCHNITT 4

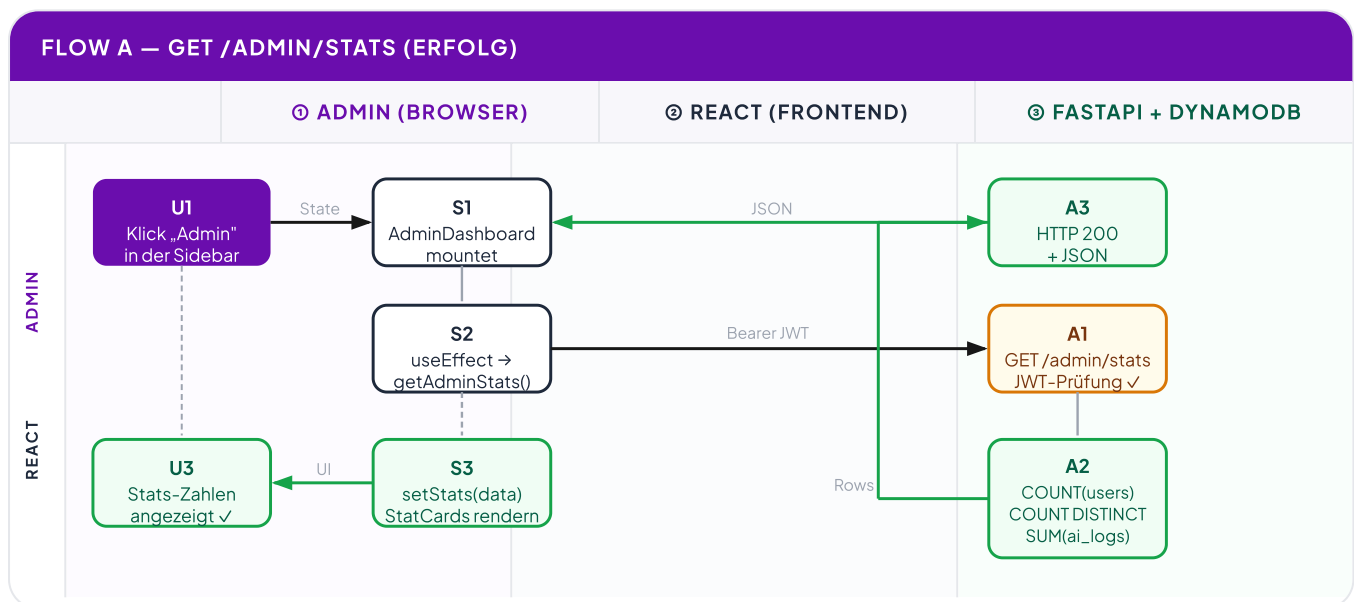
Swimlane-Ablaufdiagramm

Zwei Flows dokumentieren den Admin-Datenpfad vollständig: Flow A zeigt den Erfolgsfall (Stats laden), Flow B den Fehlerfall (fehlende Admin-Rechte). Drei Lanes: Admin-Browser, React-Frontend, FastAPI + DynamoDB.



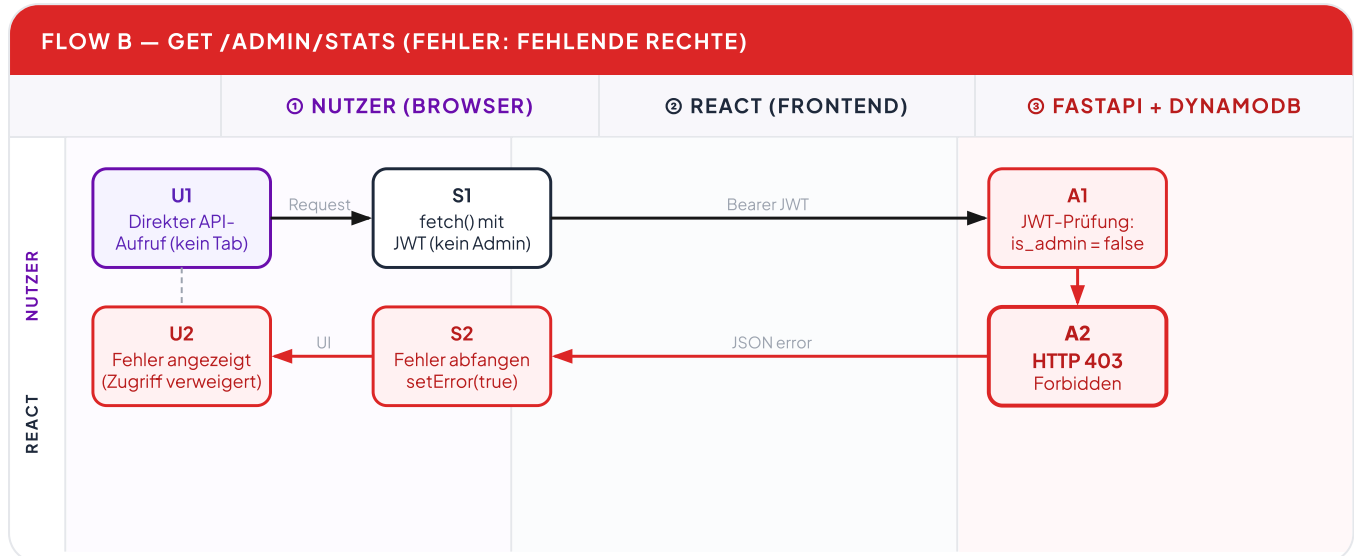
4.1 Flow A — Stats-Übersicht laden (ADM-00)

Erfolgsfall: Admin klickt auf „Admin“ in der Sidebar, das Dashboard mountet, lädt via useEffect die Stats und rendert die StatCards.



4.2 Flow B — Zugriff verweigert (HTTP 403 / 401)

Fehlerfall: Ein Nutzer ohne Admin-Rechte versucht, den Endpunkt aufzurufen. FastAPI antwortet mit HTTP 403. Das Frontend fängt den Fehler ab und zeigt eine Fehlermeldung; der Admin-Tab ist im Normalfall gar nicht sichtbar.



HTTP 403 FORBIDDEN

FastAPI gibt 403 zurück, wenn `user.is_admin == False`. Im Normalfall ist der Admin-Tab im Frontend für solche Nutzer nicht gerendert; dieser Fall tritt nur bei direktem API-Zugriff ein.

HTTP 401 UNAUTHORIZED

Fehlendes oder abgelaufenes JWT führt zu 401. Das React-Frontend leitet den Nutzer zurück zur Anmeldeseite (Auth-Dialog D-08).

ABSCHNITT 5.1

ADM-00 Stats Overview

Einzigste Ansicht des Admin-Bereichs. Vier StatCards zeigen die wichtigsten Plattformkennzahlen; eine AI-Usage-Card fasst die KI-Nutzungsstatistik zusammen. Alle Werte werden beim ersten Rendern über `GET /admin/stats` geladen.

DIALOG ADM-00 · STATS OVERVIEW

GATED · ADMIN

Reflecta

NAVIGATION

Journal

Goals

Calendar

Analytics

Admin

nur if is_admin === true

Admin Dashboard

Plattform-Statistiken · GET /admin/stats

2

ICN

Total Users

42

stats.total_users

3

ICN

Active Users (7d)

17

stats.active_users_7d

4

ICN

Total Entries

318

stats.total_entries

5

ICN

AI Calls Today

24

stats.ai_calls_today

6

AI USAGE CARD (ADM-00.5)

Total Calls

1247

stats.total_ai_calls

Success Rate (%)

98.4%

stats.ai_success_rate

Total Tokens

412 580

stats.ai_tokens_total (in + out)

Initialer Ladevorgang — useEffect → getAdminStats()

Während des Requests werden alle Werte als „-“ oder Skeleton dargestellt; setStats() ersetzt sie nach erfolgreicher Antwort.

DATENQUELLE: GET /admin/stats · SCHUTZ: get_current_admin (is_admin === true)

Keine weiteren Tabs oder Sub-Dialoge in diesem Bereich (siehe Dialogliste § 2).

CALLOUTS

1

Sidebar-Tab „Admin“ (ADM-00) — nur für is_admin-Nutzer sichtbar; aktiver Tab ist lila hinterlegt.

3

StatCard Active Users (ADM-00.2) — Nutzer mit Aktivität der letzten 7 Tage aus stats.active_users_7d.

5

StatCard AI Calls Today (ADM-00.4) — KI-Aufrufe am heutigen Datum aus stats.ai_calls_today.

2

StatCard Total Users (ADM-00.1) — Gesamtzahl registrierter Nutzer aus stats.total_users.

4

StatCard Total Entries (ADM-00.3) — Gesamte Journaleinträge über alle Konten aus stats.total_entries.

6

AI Usage Card (ADM-00.5) — Total Calls, Success Rate (%) und Total Tokens in einer Zeile.

Reflecta · Fachkonzept Admin-Bereich

09